UNIVERSITY OF GHANA

ACCRA GHANA

SCHOOL OF ENGINEERING SCIENCES

DEPARTMENT OF COMPUTER ENGINEERING

CPEN 412 - MOBILE AND WEB SOFTWARE ARCHITECTURE

Lab Report: National Emergency Response and Dispatch Coordination Platform Architecture, Database Design and API Definitions.

Contributors

Daniel Agyin Manford - 11015506

Kudiabor Jonathan Kwabena Ewenam - 11254301

Table of Contents

1. System Architecture Overview

The Emergency Response System is built as a microservice architecture composed of four core services, each owning its own MongoDB database. All client traffic is routed through an nginx API Gateway, which handles TLS termination and path-based proxying. Real-time vehicle tracking is delivered via Socket.io WebSockets, and asynchronous event propagation between services is handled by RabbitMQ topic exchanges.

The architecture diagram below illustrates the full system topology, including the frontend, API gateway, microservices, databases, and message broker.

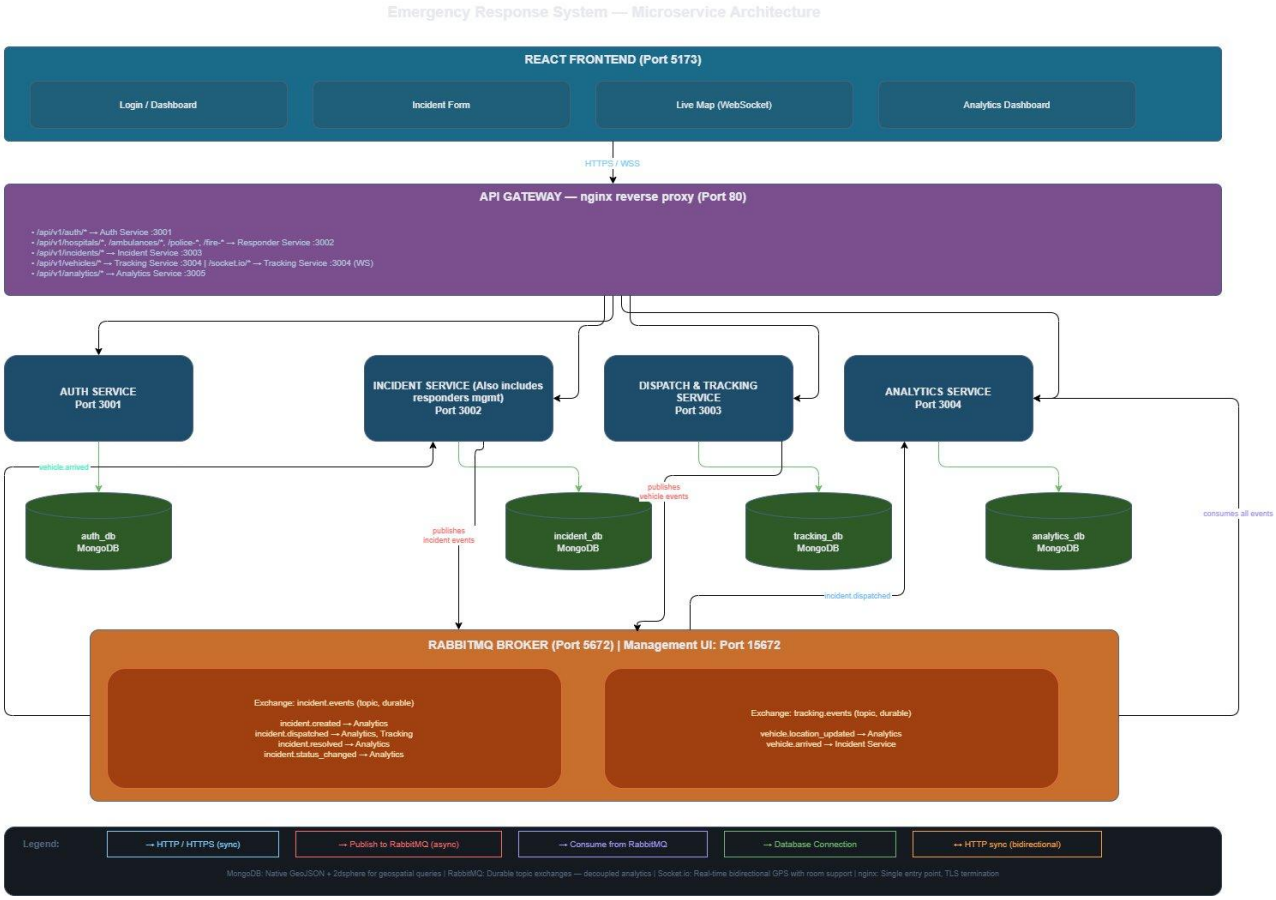

Figure 1: Emergency Response System — Microservice Architecture

2. Service Responsibilities

The following table summarises the ownership and responsibilities of each microservice.

#	Service	Port	Owns	Key Responsibilities
1	Auth Service	3001	Users, refresh tokens	Registration, login, JWT issuance, role enforcement
2	Emergency Incident Service	3002	Incidents, hospitals, ambulances, police stations, fire stations, officers, personnel	Incident creation, nearest-responder geospatial selection, responder CRUD, incident status lifecycle, event publishing
3	Dispatch & Tracking Service	3003	Dispatch records, vehicle location history, live positions	Accept GPS pings, maintain live positions, broadcast via WebSocket, detect arrival, publish vehicle events
4	Analytics & Monitoring Service	3004	Incident snapshots, response time summaries, capacity logs	Consume events, aggregate response times, resource utilisation, incident stats by region and type

Note: Responder Management was merged into the Emergency Incident Service. Incident creation and nearest-responder selection are tightly coupled - the service queries its own local models instead of making an HTTP call to a separate service. This reduces latency and eliminates an unnecessary network dependency, and aligns with the four-service requirement.

3. Inter-Service Communication

Services communicate via two mechanisms: synchronous HTTP calls using axios for operations that require an immediate response, and asynchronous RabbitMQ topic exchange events for fire-and-forget notifications.

Synchronous HTTP calls:

- Auth validation: All services validate the JWT access token using a shared secret environment variable.
- Nearest responder: The Emergency Incident Service performs a $near geospatial query internally on its own ambulance/station collections - no external HTTP call is required.

Asynchronous RabbitMQ events:

- Incident Service publishes: incident.created, incident.dispatched, incident.resolved, incident.status_changed
- Tracking Service publishes: vehicle.location_updated, vehicle.arrived

- Analytics Service subscribes to all events for aggregation.
- Tracking Service subscribes to incident.dispatched to create dispatch records.
- Incident Service subscribes to vehicle.arrived to auto-update status to in_progress.

4. Database Design

Each microservice owns its own MongoDB database. No service reads directly from another service's database. MongoDB is used across all services for its native GeoJSON and 2dsphere index support, which is critical for geospatial nearest-responder queries.

Important: GeoJSON stores coordinates as [longitude, latitude] - the reverse of the common convention. All coordinates must be stored and queried in this order.

4.1 Auth Service - auth_db

Collection: users

Field	Type	Description
_id	ObjectId	Primary key
name	String	Full name
email	String	Unique, indexed
passwordHash	String	bcrypt hash
role	String (enum)	system_admin | hospital_admin | police_admin | fire_admin | ambulance_driver
stationId	ObjectId (optional)	Links admin/driver to their hospital, station, or ambulance in Responder Service
isActive	Boolean	Default: true
createdAt / updatedAt	Date	Timestamps

Indexes: email (unique)

Collection: refresh_tokens

Field	Type	Description
_id	ObjectId	Primary key
userId	ObjectId	Ref to users._id
token	String	Hashed refresh token — unique index
expiresAt	Date	TTL index — auto-deletes expired tokens
createdAt	Date	Timestamp

4.2 Emergency Incident Service - incident_db

This service consolidates both incident management and responder management into a single database. Collections for hospitals, ambulances, police stations, fire stations, officers, and personnel all reside here alongside the incidents collection.

Collections: hospitals, ambulances, police_stations, fire_stations, police_officers, fire_personnel

Field	Type	Description
_id	ObjectId	Primary key
name	String	Name of the unit or station
address / region	String	Physical address and region
location	GeoJSON Point	Coordinates [longitude, latitude] — 2dsphere indexed
status	String (enum)	active | inactive | full (hospitals) | available | dispatched | out_of_service (ambulances)
totalBeds / availableBeds	Number	Hospital capacity (hospitals only)
vehicleNumber / badgeNumber	String	Unit identifier (ambulances, officers, personnel)
hospitalId / stationId	ObjectId	Parent reference linking ambulance → hospital, officer → station
driverId / userId	ObjectId	Auth user reference for drivers and officers
isOnDuty	Boolean	Duty status (officers and personnel only)
currentLocation	GeoJSON Point	Live location for ambulances — 2dsphere indexed
createdAt / updatedAt	Date	Timestamps

Collection: incidents

Field	Type	Description
_id	ObjectId	Primary key
citizenName / citizenPhone	String	Reporter details
incidentType	String (enum)	medical | fire | crime | accident | other
location	GeoJSON Point	Incident coordinates — 2dsphere indexed
address	String	Human-readable address from reverse geocode
notes	String	Free-text description
createdBy	ObjectId	Auth user ID of reporting system admin
assignedUnit	Object	{ unitId, unitType, unitName, hospitalId } — denormalised for display
status	String (enum)	created | dispatched | in_progress | resolved | cancelled

statusHistory	Array	Array of { status, changedAt, changedBy } — full audit trail
dispatchedAt / resolvedAt	Date	Key timestamps
responseTimeMinutes	Number	Computed on resolve
createdAt / updatedAt	Date	Timestamps

Indexes: location (2dsphere), status, incidentType, createdAt (descending)

4.3 Dispatch & Tracking Service - tracking_db

Collection: dispatch_records

Field	Type	Description
_id	ObjectId	Primary key
incidentId	ObjectId	Reference to Incident Service incident ID
vehicleId	ObjectId	Ambulance, police vehicle, or fire truck ID
vehicleType	String (enum)	ambulance | police_vehicle | fire_truck
driverId	ObjectId	Auth user ID of responding driver or officer
dispatchedAt / arrivedAt / completedAt	Date	Key lifecycle timestamps
status	String (enum)	en_route | arrived | completed | cancelled

Collection: vehicle_locations

Field	Type	Description
_id	ObjectId	Primary key
vehicleId / vehicleType	ObjectId / String	Vehicle reference
incidentId	ObjectId	Active incident being responded to
location	GeoJSON Point	2dsphere indexed
speed	Number	Optional, km/h
heading	Number	Optional, degrees
recordedAt	Date	GPS ping timestamp — compound index with vehicleId; optional TTL (7 days)

Collection: live_positions

One document per vehicle, upserted on each GPS ping for fast current-location lookups.

Field	Type	Description
vehicleId	ObjectId	Unique index
vehicleType / incidentId	String / ObjectId	Active context
location	GeoJSON Point	2dsphere indexed
lastUpdated	Date	Timestamp of most recent ping

4.4 Analytics Service - analytics_db

Field	Type	Description
incident_snapshots	Collection	Denormalised per-incident records populated from RabbitMQ events. Fields: incidentId, incidentType, region, latitude, longitude, status, assignedUnitType, responseTimeMinutes, key dates.
hospital_capacity_logs	Collection	Time-series snapshots of hospital bed occupancy: hospitalId, name, totalBeds, availableBeds, occupancyPercent, recordedAt.
response_time_summaries	Collection	Pre-aggregated for fast dashboard queries. Fields: period (e.g. 2025-03), periodType (day|month), incidentType, region, avgResponseTimeMinutes, totalIncidents, resolvedIncidents, computedAt.

5. Message Queue Design

RabbitMQ is used for asynchronous, event-driven communication between services where the publisher does not need an immediate response and multiple services may need to react to the same event. Direct HTTP (axios) is used only when an immediate response is required (for example, JWT validation).

Broker configuration: Host: rabbitmq (Docker service name), AMQP Port: 5672, Management UI: Port 15672.

5.1 Exchange Definitions & Queue Bindings

Two durable topic exchanges are declared:

- incident.events — all lifecycle events of emergency incidents (type: topic, durable: true)
- tracking.events — vehicle location and dispatch status events (type: topic, durable: true)

Queues bound to incident.events

Queue Name	Binding Key	Consumer Service	Description
analytics.incident.created	incident.created	Analytics Service	Record new incident snapshot
analytics.incident.dispatched	incident.dispatched	Analytics Service	Log dispatch time
analytics.incident.resolved	incident.resolved	Analytics Service	Compute response time
analytics.incident.status	incident.status_changed	Analytics Service	General status tracking
tracking.incident.dispatched	incident.dispatched	Tracking Service	Create dispatch record

Queues bound to tracking.events

Queue Name	Binding Key	Consumer Service	Description
analytics.vehicle.location	vehicle.location_updated	Analytics Service	Aggregate location data
incident.vehicle.arrived	vehicle.arrived	Incident Service	Auto-update incident to in_progress

5.2 Event Message Structures

All messages are JSON with ISO 8601 UTC timestamps. Key event payloads are described below.

Event Type	Published By	Consumed By	Key Payload Fields
incident.created	Incident Service	Analytics Service	incidentId, incidentType, latitude, longitude, address, region, createdBy, createdAt
incident.dispatched	Incident Service	Analytics Service, Tracking Service	incidentId, incidentType, latitude, longitude, assignedUnit { unitId, unitType, unitName, hospitalId, driverId }, dispatchedAt
incident.status_changed	Incident Service	Analytics Service	incidentId, previousStatus, newStatus, changedBy, changedAt
incident.resolved	Incident Service	Analytics Service	incidentId, incidentType, region, createdAt, dispatchedAt, resolvedAt, responseTimeMinutes, assignedUnitType
vehicle.location_updated	Tracking Service	Analytics Service	vehicleId, vehicleType, incidentId, latitude, longitude, speed, recordedAt
vehicle.arrived	Tracking Service	Incident Service	vehicleId, vehicleType, incidentId, arrivedAt

5.3 End-to-End Communication Flow

The diagram below shows the complete flow from incident submission through dispatch, GPS tracking, vehicle arrival, and final resolution.

Communication Flow: New Incident End-to-End

Emergency Response System — RabbitMQ Message Queue & Service Orchestration

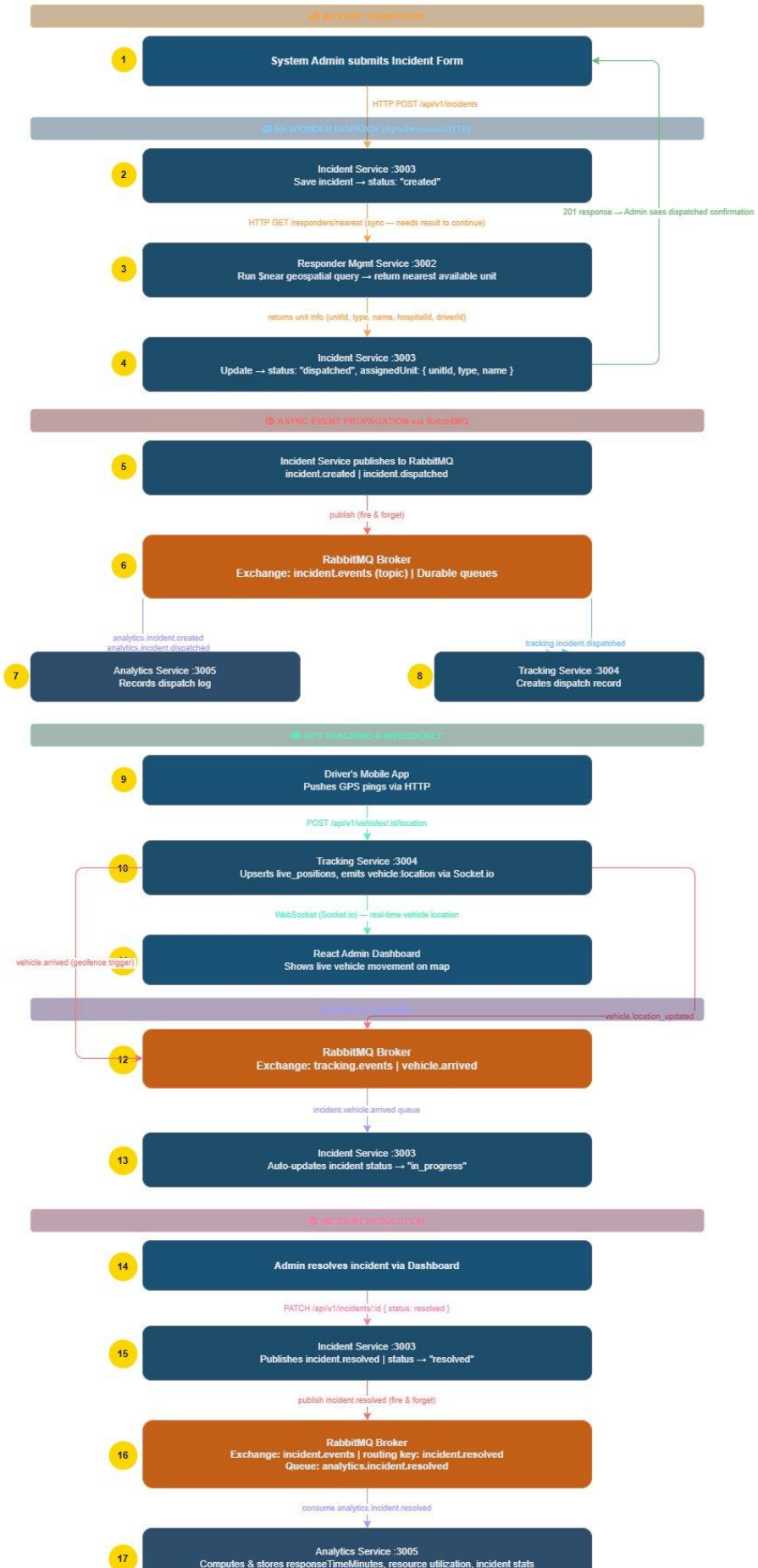

Figure 2: New Incident Communication Flow - End-to-End

The flow proceeds in six phases:

1. Incident Submission - Admin submits the incident form via the React frontend.
2. Synchronous Dispatch - Incident Service saves the incident, queries the nearest available responder via a $near geospatial query, updates status to dispatched, and returns a 201 response to the admin.
3. Async Event Propagation - Incident Service publishes incident.created and incident.dispatched to RabbitMQ. Analytics Service records the dispatch log; Tracking Service creates the dispatch record.
4. GPS Tracking & WebSocket - The driver's app pushes GPS pings to Tracking Service, which upserts live_positions and emits vehicle:location events via Socket.io to the admin dashboard.
5. Vehicle Arrival - Tracking Service publishes vehicle.arrived; Incident Service consumes this and auto-updates the incident status to in_progress.
6. Resolution - Admin resolves the incident; Incident Service publishes incident.resolved; Analytics Service computes and stores responseTimeMinutes and resource utilisation.

6. API Definitions

All APIs are versioned under /api/v1/. All routes except /auth/login and /auth/register require a valid JWT in the Authorization: Bearer <token> header.

Standard response envelopes:

- Success: { "success": true, "data": { ... } }
- Error: { "success": false, "error": { "code": "ERROR_CODE", "message": "Human readable message" } }

6.1 Auth Service (Port 3001)

Method	Endpoint	Auth	Role	Description
POST	/api/v1/auth/register	No	-	Register a new user
POST	/api/v1/auth/login	No	-	Login, returns JWT + refresh token
POST	/api/v1/auth/refresh-token	No	-	Exchange refresh token for new access token
POST	/api/v1/auth/logout	Yes	Any	Revoke refresh token
GET	/api/v1/auth/profile	Yes	Any	Get own profile
PUT	/api/v1/auth/profile	Yes	Any	Update own profile
GET	/api/v1/auth/users	Yes	system_admin	List all users
GET	/api/v1/auth/users/:id	Yes	system_admin	Get user by ID
PUT	/api/v1/auth/users/:id/status	Yes	system_admin	Activate/deactivate user
DELETE	/api/v1/auth/users/:id	Yes	system_admin	Delete user

Key Request/Response Notes

POST /api/v1/auth/register - Request body: name, email, password, role, stationId (optional). Returns 201 with user object.

POST /api/v1/auth/login - Returns accessToken (expires 15 min), refreshToken (expires 7 days), and user object.

POST /api/v1/auth/refresh-token - Request body: { refreshToken }. Returns new accessToken.

6.2 Emergency Incident Service (Port 3002)

Hospitals

Method	Endpoint	Auth	Role	Description
POST	/api/v1/hospitals	Yes	system_admin	Register a new hospital
GET	/api/v1/hospitals	Yes	Any	List all hospitals
GET	/api/v1/hospitals/:id	Yes	Any	Get hospital details
PUT	/api/v1/hospitals/:id	Yes	hospital_admin, system_admin	Update hospital info
PUT	/api/v1/hospitals/:id/capacity	Yes	hospital_admin	Update bed availability
DELETE	/api/v1/hospitals/:id	Yes	system_admin	Remove hospital

Ambulances

Method	Endpoint	Auth	Role	Description
POST	/api/v1/ambulances	Yes	hospital_admin, system_admin	Register ambulance
GET	/api/v1/ambulances	Yes	Any	List all ambulances
GET	/api/v1/ambulances/:id	Yes	Any	Get ambulance details
PUT	/api/v1/ambulances/:id/status	Yes	hospital_admin, ambulance_driver	Update availability status
GET	/api/v1/ambulances/available	Yes	Any	List available ambulances with location

Police Stations

Method	Endpoint	Auth	Role	Description
POST	/api/v1/police-stations	Yes	system_admin	Register police station
GET	/api/v1/police-stations	Yes	Any	List all stations
GET	/api/v1/police-stations/:id	Yes	Any	Get station details
PUT	/api/v1/police-stations/:id	Yes	police_admin, system_admin	Update station info
PUT	/api/v1/police-stations/:id/status	Yes	police_admin	Update availability
POST	/api/v1/police-stations/:id/officers	Yes	police_admin	Add officer to station
GET	/api/v1/police-stations/:id/officers	Yes	police_admin, system_admin	List officers

Fire Stations

Method	Endpoint	Auth	Role	Description
POST	/api/v1/fire-stations	Yes	system_admin	Register fire station
GET	/api/v1/fire-stations	Yes	Any	List all stations
GET	/api/v1/fire-stations/:id	Yes	Any	Get station details
PUT	/api/v1/fire-stations/:id	Yes	fire_admin, system_admin	Update station info
PUT	/api/v1/fire-stations/:id/status	Yes	fire_admin	Update availability
POST	/api/v1/fire-stations/:id/personnel	Yes	fire_admin	Add personnel
GET	/api/v1/fire-stations/:id/personnel	Yes	fire_admin, system_admin	List personnel

Incidents

Method	Endpoint	Auth	Role	Description
POST	/api/v1/incidents	Yes	system_admin	Create new incident + auto-dispatch
GET	/api/v1/incidents	Yes	system_admin	List all incidents (paginated)
GET	/api/v1/incidents/open	Yes	system_admin	List open/active incidents
GET	/api/v1/incidents/:id	Yes	system_admin	Get incident details
PUT	/api/v1/incidents/:id/status	Yes	system_admin	Manually update status
PUT	/api/v1/incidents/:id/assign	Yes	system_admin	Reassign to different unit
GET	/api/v1/incidents/stats	Yes	system_admin	Summary counts by status

Nearest Responder

Method	Endpoint	Auth	Role	Description
GET	/api/v1/responders/nearest	Yes	Any	Find nearest available responder — query params: lat, lng, type (medical|fire|crime), maxDistance (metres, default 50000)

Response includes: unitId, unitType, unitName, hospitalId, hospitalName, distanceMeters, estimatedMinutes.

6.3 Dispatch & Tracking Service (Port 3003)

HTTP Endpoints

Method	Endpoint	Auth	Role	Description
POST	/api/v1/vehicles/register	Yes	system_admin	Register a vehicle for tracking
GET	/api/v1/vehicles	Yes	Any	List all tracked vehicles
GET	/api/v1/vehicles/:id/location	Yes	Any	Get current location of a vehicle
POST	/api/v1/vehicles/:id/location	Yes	ambulance_driver	Push a GPS location update
GET	/api/v1/dispatches	Yes	system_admin	List all dispatch records
GET	/api/v1/dispatches/:incidentId	Yes	system_admin	Get dispatch for an incident
PUT	/api/v1/dispatches/:id/status	Yes	ambulance_driver, system_admin	Update dispatch status

POST /api/v1/vehicles/:id/location request body: latitude, longitude, speed (optional), heading (optional), incidentId.

WebSocket Events (Socket.io)

Clients connect to: ws://localhost:3003

Client → Server

Event	Payload	Description
subscribe:incident	{ incidentId }	Subscribe to live updates for a specific incident
subscribe:all	{}	Subscribe to all vehicle movements (admin map view)
unsubscribe:incident	{ incidentId }	Unsubscribe from a specific incident

Server → Client

Event	Payload	Description
vehicle:location	{ vehicleId, incidentId, lat, lng, timestamp }	Real-time position update
dispatch:status_changed	{ incidentId, status, timestamp }	Dispatch status update
vehicle:arrived	{ vehicleId, incidentId, arrivedAt }	Vehicle arrived at scene

6.4 Analytics Service (Port 3004)

Method	Endpoint	Auth	Role	Description
GET	/api/v1/analytics/response-times	Yes	Any	Average response times by type/period — params: from, to, incidentType, periodType (day|month)
GET	/api/v1/analytics/incidents-by-region	Yes	Any	Incident counts by region and type

GET	/api/v1/analytics/resource-utilization	Yes	Any	Hospital capacity, vehicle deployment stats
GET	/api/v1/analytics/incidents-by-type	Yes	Any	Breakdown by incident type
GET	/api/v1/analytics/top-responders	Yes	Any	Most frequently deployed responders
GET	/api/v1/analytics/overview	Yes	Any	Dashboard summary stats

Overview response includes: totalIncidents, openIncidents, resolvedToday, avgResponseTimeMinutes, incidentsByType (medical, fire, crime, accident).

6.5 API Gateway Routes (nginx)

Path Pattern	Upstream Service
/api/v1/auth/	auth-service:3001
/api/v1/hospitals/	incident-service:3002
/api/v1/ambulances/	incident-service:3002
/api/v1/police-stations/	incident-service:3002
/api/v1/fire-stations/	incident-service:3002
/api/v1/responders/	incident-service:3002
/api/v1/incidents/	incident-service:3002
/api/v1/vehicles/	tracking-service:3003
/api/v1/dispatches/	tracking-service:3003
/socket.io/	tracking-service:3003 (WebSocket upgrade)
/api/v1/analytics/	analytics-service:3004

7. Technology Decisions Rationale

Decision	Rationale
MongoDB for all services	Native GeoJSON and 2dsphere index support enables geospatial nearest-responder queries without any extension. The flexible schema suits incidents (notes, varying fields per type) and analytics (varying metrics). All services share the same database engine while maintaining database-level isolation per microservice.
Merging Responder Management into Emergency Incident Service	Nearest-responder selection is a core part of incident creation - the two are logically coupled. Merging eliminates a synchronous HTTP call on the critical path, reduces infrastructure complexity (one fewer service and database), and aligns with the four-service requirement.
RabbitMQ over direct HTTP for events	Without a message queue, Analytics Service would need to be called directly from multiple services, creating tight coupling. RabbitMQ decouples publishers from consumers: if Analytics is down, incidents still succeed. Events are naturally fire-and-forget, and RabbitMQ persistence ensures events are not lost if a consumer restarts.
Socket.io for real-time tracking	Provides real-time bidirectional communication: drivers push GPS location and the admin dashboard receives it instantly. Room support allows admins to subscribe to specific incidents only. Automatic fallback to long-polling if WebSocket is unavailable, with native Node.js support.
nginx as API Gateway	Single entry point for the frontend eliminates CORS issues. Handles TLS termination in production. Lightweight, widely deployed, and straightforward to configure without building a custom Express gateway.

8. Docker Compose Service Map

The following services are defined in docker-compose.yml:

Service	Port	Description
rabbitmq	5672, 15672	Message broker — AMQP and management UI
auth-db	-	MongoDB instance for auth_db
auth-service	3001	Identity and authentication microservice
incident-db	-	MongoDB instance for incident_db
incident-service	3002	Emergency Incident Service (includes responder management)
tracking-db	-	MongoDB instance for tracking_db
tracking-service	3003	Dispatch & Tracking Service
analytics-db	-	MongoDB instance for analytics_db
analytics-service	3004	Analytics & Monitoring Service
frontend	5173	React / Vite dev server (optional in docker-compose)
nginx	80	API Gateway — reverse proxy